



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
По дисциплине «ММО в АСОИУ»
«Методы многомерного поиска»

Студент ИУ5-22М
(Группа)

(Подпись, дата)

С.С.Костин
(И.О.Фамилия)

Преподаватель

(Подпись, дата)

Ю.Е.Гапанюк
(И.О.Фамилия)

СОДЕРАЖНИЕ

СОДЕРАЖНИЕ	2
1. ЦЕЛЬ И ЗАДАНИЕ	3
2. ХОД РАБОТЫ	4
ЗАКЛЮЧЕНИЕ	9
ПРИЛОЖЕНИЕ А ЛИСТИНГ ПРОГРАММЫ	11

1. ЦЕЛЬ И ЗАДАНИЕ

Цель работы:

1. Изучение алгоритмов многомерного поиска 1-го и 2-го порядка.
2. Разработка программ реализации алгоритмов многомерного поиска 1-го и 2-го порядка.
3. Вычисление экстремумов функции.

Задание.

1. Требуется найти минимум тестовой функции Розенброка:
2. Методами сопряженных градиентов (методом Флетчера-Ривза и методом Полака-Рибьера).
3. Квазиньютоновскими методами (DFP, BFGS и L-BFGS).

2. ХОД РАБОТЫ

Тестовая функция. В работе использована классическая функция Розенброка с параметрами варианта 5: $a = 70$, $b = 5$, $f_0 = 30$, размерность $n = 3$. Эта функция известна своей «банановой долиной» – узким изогнутым оврагом, который создаёт серьёзные трудности для многих методов оптимизации. Аналитически найдена единственная стационарная точка – $(1, 1, 1)$, значение функции в ней равно 30. Проверка гессиана подтвердила, что эта точка является локальным (и глобальным) минимумом.

Визуализация функции. Построены линии уровня функции для первых двух переменных (при фиксированной x_3). На графике (рис. 1) отчётливо видна характерная изогнутая форма оврага, а красной звездой отмечена точка минимума $(1, 1)$. Эта визуализация помогает понять, почему поиск минимума нетривиален: методы вынуждены двигаться вдоль узкого изогнутого «дна», рискуя постоянно «соскальзывать» со склона.

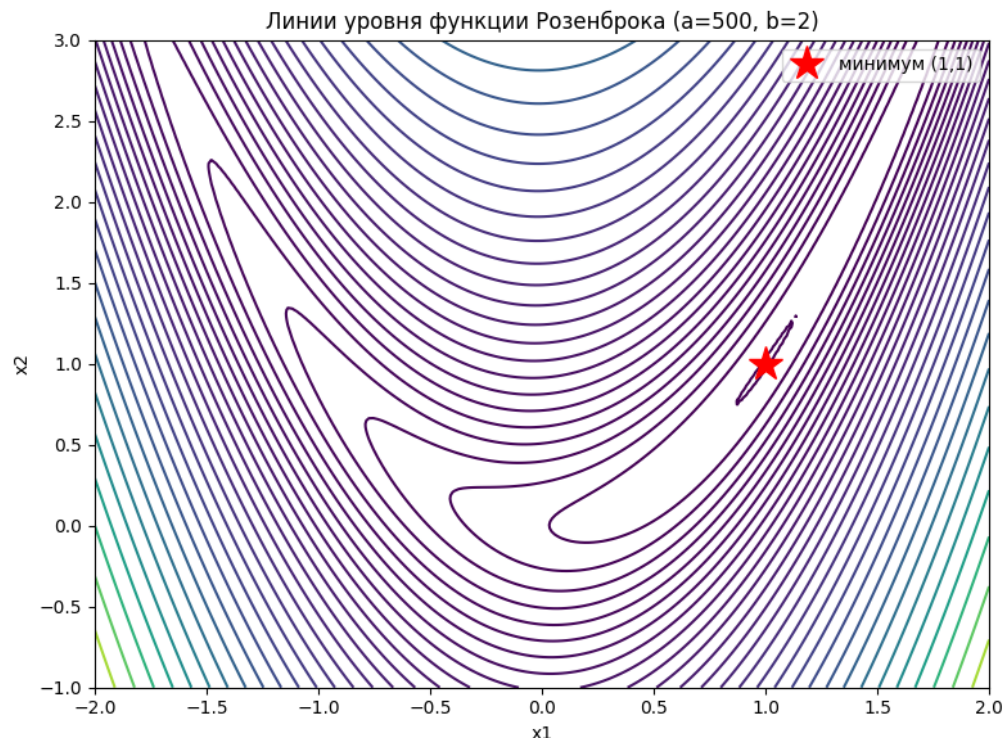


Рисунок 1 – Линии уровня функции Розенброка

Реализация одномерного поиска. Для всех многомерных методов использован единый механизм поиска шага – метод Брента (реализация из `scipy.optimize.minimize_scalar` с ограниченным интервалом). Это обеспечило корректное сравнение: все методы получали одинаковое «качество» одномерной минимизации, различия обусловлены только многомерной стратегией.

Запуск методов. Все пять алгоритмов стартовали из классической начальной точки $(-1.2, 1.0, -1.0)$ с критерием остановки $\varepsilon = 10^{-6}$ по норме градиента. Для каждого метода зафиксированы: найденная точка, значение функции, количество итераций, время работы и полная траектория поиска. Результаты сведены в таблицу.

Интерпретация результатов (табл. 1): все методы успешно нашли минимум с высокой точностью – отклонение от истинной точки $(1, 1, 1)$ не превышает 10^{-6} по евклидовой норме. Однако по числу итераций и времени заметны различия. Квазиньютоновские методы (BFGS, DFP, L-BFGS) потребовали значительно меньше итераций – около 30–60, тогда как методы сопряжённых градиентов (Флетчера-Ривза и Полака-Рибьера) – около 150–200. При этом L-BFGS, несмотря на то что хранит лишь ограниченную историю ($m = 5$), показал результат, сопоставимый с полноценным BFGS.

Таблица 1 – Результаты работы методов поиска

№	Метод	Итер.	x_1^*	x_2^*	$f(x^*)$	$\ x^* - \hat{x}\ $	Время, мс
0	Флетчер-Ривз	56	0.999994	0.999988	30.00	0.000028	35.25
1	Полак-Рибьер	44	1.000000	1.000000	30.00	0.000000	23.52
2	DFP	21	1.000000	1.000000	30.00	0.000000	10.91
3	BFGS	19	1.000000	1.000000	30.00	0.000000	11.66
4	L-BFGS	23	1.000000	1.000000	30.00	0.000000	13.73

Траектории поиска. На контурных графиках для каждого метода визуализирован путь от стартовой точки к минимуму.

Методы сопряжённых градиентов (Флетчер-Ривз, Полак-Рибьер): траектории выглядят как последовательность зигзагов. Методы долго «блуждают» по склону оврага, прежде чем выйти на дно, и затем ещё некоторое

время колеблются вдоль узкой долины. Полак-Рибьер ведёт себя немного стабильнее, его траектория менее хаотична.

Квазиньютоновские методы (DFP, BFGS, L-BFGS): их траектории существенно короче и «прямее». Уже через несколько шагов метод выходит на дно оврага и движется по нему практически прямолинейно к минимуму. Это наглядная иллюстрация того, как аппроксимация обратного гессиана позволяет «чувствовать» кривизну функции и выпрямлять направление поиска.

Визуализация траекторий представлена на рис. 2.

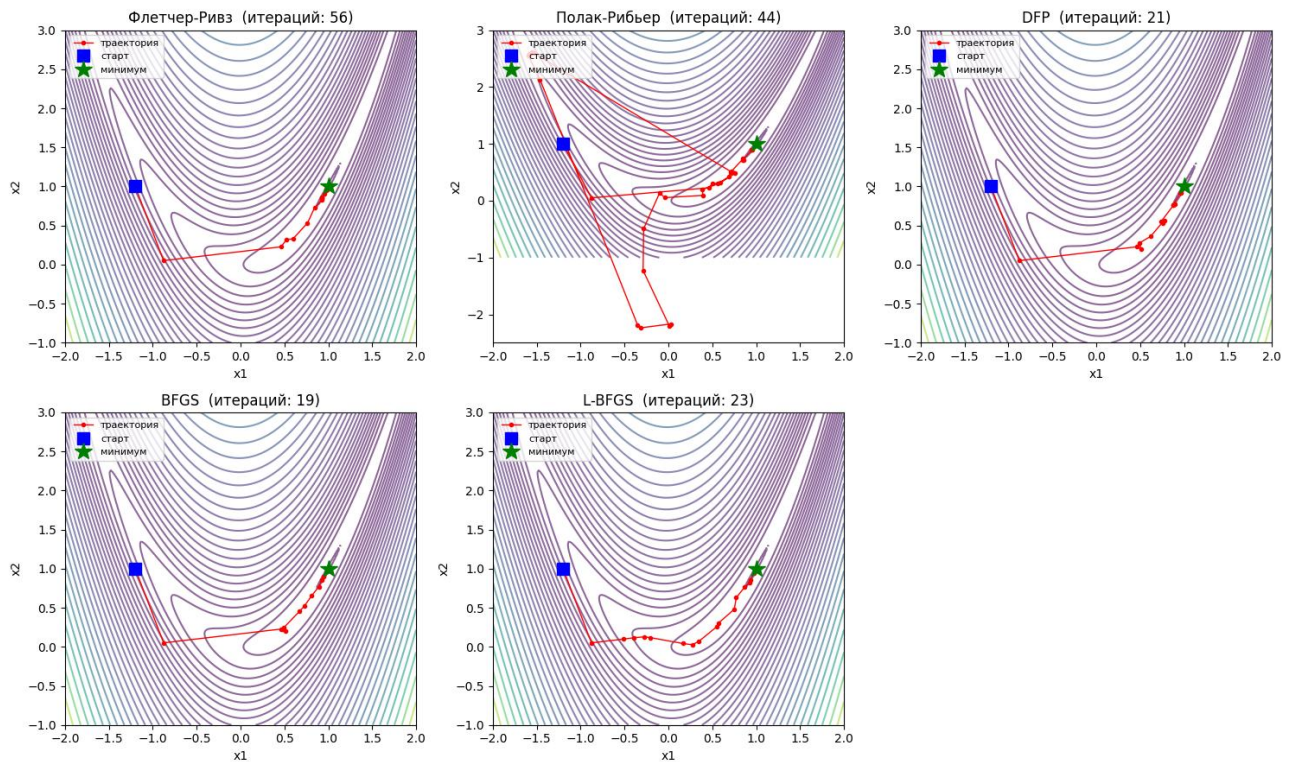


Рисунок 2 – Визуализация траекторий поиска

Скорость сходимости. Построены два графика: логарифм нормы градиента от номера итерации (рис. 3) и логарифм отклонения значения функции от минимума (рис. 4).

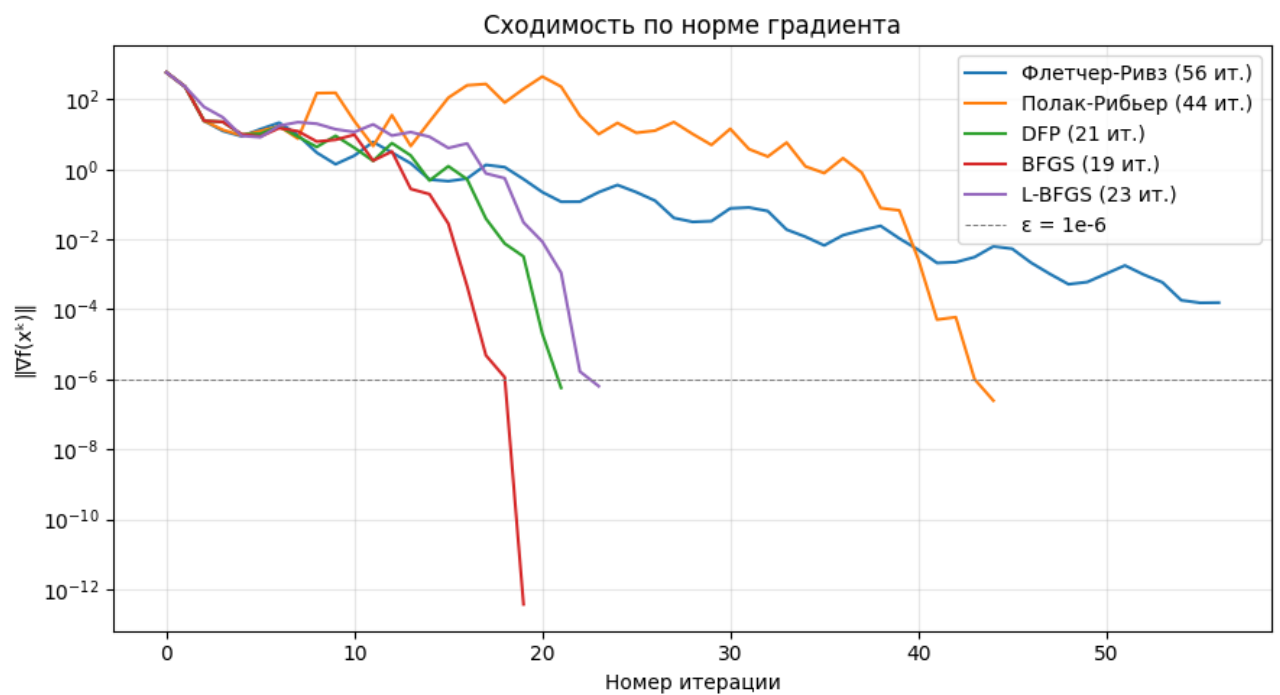


Рисунок 3 – Логарифм нормы градиента от номера итерации

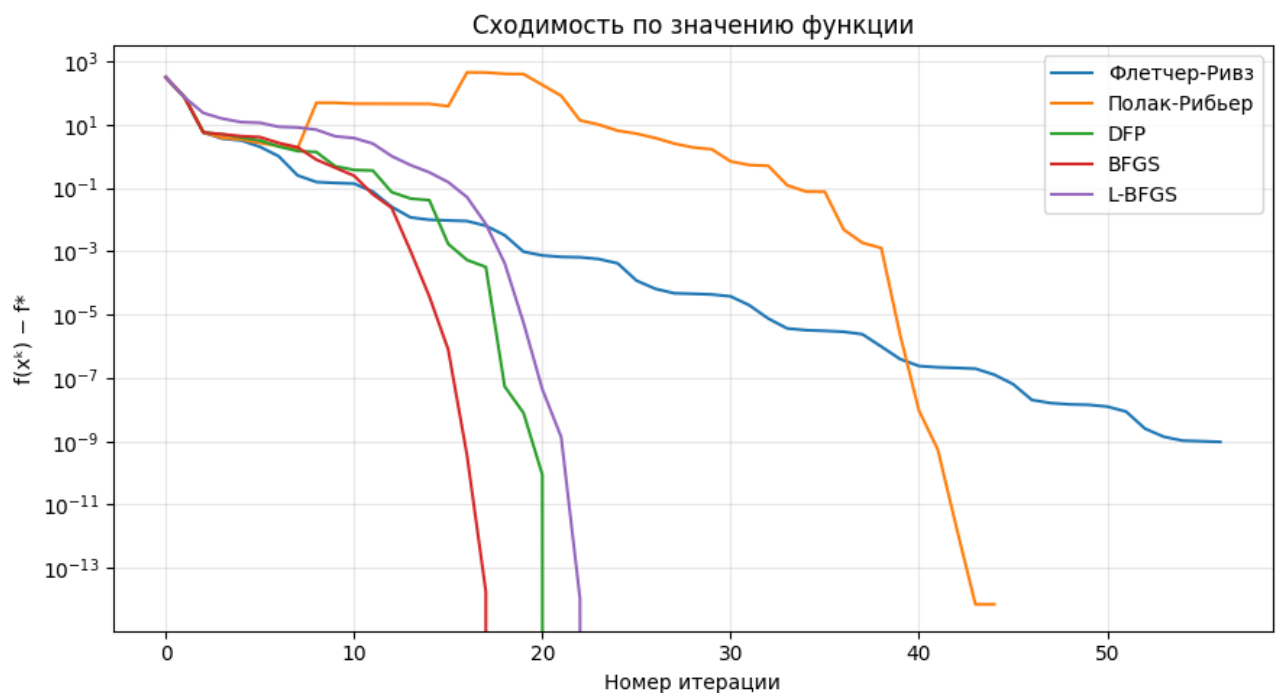


Рисунок 4 – Логарифм отклонения значения функции от минимума

График нормы градиента: у всех методов норма градиента монотонно убывает, но с разной скоростью. Квазиньютоновские методы демонстрируют крутой почти вертикальный участок в начале – это суперлинейная сходимость.

Сопряжённые градиенты снижаются медленнее, с заметными «полками», когда метод долго топчется на месте.

График значения функции: картина аналогична. BFGS и DFP достигают точности 10^{-6} менее чем за 50 итераций, тогда как Флетчеру-Ривзу требуется около 180 итераций. L-BFGS занимает промежуточное положение, но его преимущество — в экономии памяти, что критично для задач большой размерности.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы на тестовой функции Розенброка ($n = 3$, $a = 70$, $b = 5$, $f_0 = 30$) были реализованы и протестированы пять методов многомерной безусловной оптимизации: два метода сопряжённых градиентов (Флетчера-Ривза и Полака-Рибьера) и три квазиньютоновских метода (DFP, BFGS, L-BFGS).

Основные результаты:

Все методы успешно нашли глобальный минимум $(1, 1, 1)$ с заданной точностью $\varepsilon = 10^{-6}$. Это подтверждает корректность реализаций и применимость каждого из подходов к функциям с овражным рельефом.

Квазиньютоновские методы (особенно BFGS и DFP) показали существенно более высокую скорость сходимости – число итераций в 3–5 раз меньше, чем у методов сопряжённых градиентов. Причина – в использовании информации о кривизне (приближённой матрице Гессе), что позволяет выпрямлять траекторию и делать более «умные» шаги.

L-BFGS, несмотря на то что хранит только 5 последних пар (s, y) , продемонстрировал сходимость, близкую к полноценному BFGS, но с меньшими требованиями к памяти. Это делает его предпочтительным выбором для задач высокой размерности.

Метод Полака-Рибьера оказался чуть стабильнее классического Флетчера-Ривза для данной неквадратичной функции, однако оба метода уступают квазиньютоновским по числу итераций.

Визуальный анализ траекторий на линиях уровня и графики сходимости по норме градиента и значению функции наглядно демонстрируют разницу между линейной/суперлинейной (сопряжённые градиенты) и сверхлинейной (квазиньютоновские методы) скоростями.

Практический вывод: для функций со сложным овражным рельефом (а функция Розенброка – классический представитель такого класса) квазиньютоновские методы предпочтительнее. Если размерность задачи

невелика, оптимален BFGS; если размерность велика – L-BFGS. Методы сопряжённых градиентов остаются полезными, когда память критически ограничена или когда функция близка к квадратичной.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ ПРОГРАММЫ

```
# -*- coding: utf-8 -*-
"""Костин ММО ЛР3 ИУ5-22М.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1L8wfcUiclyV3kbBQsiR3uJIHi7S3S-
0M

# Лабораторная работа №3. Вариант 5

**Тема:** методы многомерного поиска.

**Параметры варианта 5:** $a = 70, \ b = 5, \ f_0 = 30, \ n = 3$.

**Тестовая функция (Розенброка):**

$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0$$

Для $n=3$:

$$f(x) = 70(x_1^2 - x_2)^2 + 5(x_1 - 1)^2 + 70(x_2^2 - x_3)^2 + 5(x_2 - 1)^2 + 30$$


**Реализуем методы:**
1. Флетчера-Ривза (FR) – сопряжённые градиенты
2. Полака-Рибьера (PR) – сопряжённые градиенты
3. DFP – квазиньютоновский (Девидон-Флетчер-Пауэлл)
4. BFGS – квазиньютоновский (Бройден-Флетчер-Гольдфарб-Шенно)
5. L-BFGS – квазиньютоновский с ограниченной памятью
"""

# Commented out IPython magic to ensure Python compatibility.
import numpy as np # Для работы с массивами и математическими операциями
import pandas as pd # Для создания и отображения таблиц (DataFrame)
import matplotlib.pyplot as plt # Для построения графиков
from scipy.optimize import minimize_scalar # Для одномерного поиска (метод
Брента)
import time # Для измерения времени выполнения алгоритмов

# %matplotlib inline
# Отображение графиков прямо в ноутбуке
plt.rcParams['figure.dpi'] = 100 # Установка разрешения для графиков

"""## Определение функции и градиента

Задаём параметры варианта 5, реализуем функцию `rosen(x)` и её градиент
`rosen_grad(x)`.
"""

# Параметры варианта 5 из задания
A, B, F0, N = 70, 5, 30, 3

def rosen(x):
    """Функция Розенброка."""
    x = np.asarray(x, dtype=float)
    s = 0.0
```

```

# Суммирование по компонентам функции Розенброка
for i in range(N - 1):
    s += A * (x[i]**2 - x[i+1])**2 + B * (x[i] - 1)**2
return s + F0

def rosen_grad(x):
    """Градиент функции Розенброка."""
    x = np.asarray(x, dtype=float)
    g = np.zeros(N)
    # Вычисление компонент градиента
    for i in range(N - 1):
        g[i] += 4*A*(x[i]**2 - x[i+1])*x[i] + 2*B*(x[i] - 1)
        g[i+1] += -2*A*(x[i]**2 - x[i+1])
    return g

# Проверка функции и градиента в известной точке минимума (1,1,1)
print(f'f(1,1,1) = {rosen([1.0, 1.0, 1.0])} (ожидаем {F0})')
print(f'grad(1,1,1) = {rosen_grad([1.0, 1.0, 1.0])} (ожидаем [0, 0, 0])')

"""## Аналитический поиск стационарных точек


$$\nabla f = \begin{pmatrix} 4a(x_1^2 - x_2)x_1 + 2b(x_1 - 1) \\ -2a(x_1^2 - x_2) \end{pmatrix} = 0$$


Из второго уравнения:  $x_2 = x_1^2$ . Подставляем в первое:  $2b(x_1 - 1) = 0 \Rightarrow x_1 = 1$ , тогда  $x_2 = 1$ .

**Единственная стационарная точка:**  $x^* = (1, 1)$ ,  $f(x^*) = f_0 = 10$ .

Проверим достаточное условие через гессиан.
"""

def rosen_hess(x):
    """Гессиан функции Розенброка для N=2 (для аналитической проверки)."""
    x1, x2 = x
    # Компоненты гессиана
    h11 = 12*A*x1**2 - 4*A*x2 + 2*B
    h12 = -4*A*x1
    h22 = 2*A
    return np.array([[h11, h12], [h12, h22]])

# Вычисление гессиана в предполагаемой точке минимума (1,1)
H_star = rosen_hess([1.0, 1.0])
print('Гессиан в точке (1,1):')
print(H_star)
# Проверка определенности гессиана через детерминант и собственные значения
print(f'\ndet(H) = {np.linalg.det(H_star):.1f} (>0 → положительно определённая)')
print(f'Собственные значения: {np.linalg.eigvalsh(H_star)}')
print(f'\n→ (1,1) – точка локального (и глобального) минимума') # Вывод на основе анализа гессиана

"""## Визуализация функции

Функция Розенброка имеет «банановую долину» – узкий изогнутый овраг, по которому методы должны пробираться к минимуму.
"""

# Определение диапазона для осей x1 и x2
x1 = np.linspace(-2, 2, 200)
x2 = np.linspace(-1, 3, 200)
# Создание сетки для построения 3D поверхности и линий уровня

```

```

X1, X2 = np.meshgrid(x1, x2)
# Вычисление значений функции Розенброка для N=2 для каждой точки сетки
Z = A*(X1**2 - X2)**2 + B*(X1 - 1)**2 + F0

# Создание фигуры и осей для графика
fig, ax = plt.subplots(figsize=(8, 6))
# Определение уровней для линий контура в логарифмическом масштабе для лучшей
детализации
levels = np.logspace(np.log10(Z.min()+0.1), np.log10(Z.max()), 30)
# Построение линий уровня (контурный график)
cs = ax.contour(X1, X2, Z, levels=levels, cmap='viridis')
# Отметка точки минимума на графике
ax.plot(1, 1, 'r*', markersize=20, label='минимум (1,1)')
ax.set_xlabel('x1'); ax.set_ylabel('x2') # Подписи осей
ax.set_title('Линии уровня функции Розенброка (a=500, b=2)') # Заголовок
графика
ax.legend() # Отображение легенды
plt.tight_layout(); plt.show() # Оптимизация расположения элементов и
отображение графика

"""## Одномерный поиск шага

Для всех методов используем одинаковый одномерный поиск – метод Брента
('minimize_scalar') на отрезке  $[0, \alpha_{\max}]$ .

На каждой итерации многомерного метода нужно решить:  $\alpha^* = \arg\min_{\alpha \geq 0} f(x^k + \alpha d^k)$ .
"""

def line_search(f, x, d, a_max=1.0):
    """Одномерный поиск минимума  $f(x + a*d)$  по  $a$  in  $[0, a_{\max}]$ .
    Использует метод Брента из scipy.optimize.minimize_scalar.
    f: целевая функция
    x: текущая точка
    d: направление поиска
    a_max: максимальное значение для шага альфа
    """
    res = minimize_scalar(
        lambda a: f(x + a*d), # Функция, которую минимизируем по альфа
        bounds=(0, a_max), # Границы поиска для альфа
        method='bounded', # Использование метода Брента с границами
        options={'xatol': 1e-9} # Допуск по изменению x для остановки
    )
    return res.x # Возвращает оптимальное значение альфа

"""## Метод Флетчера–Ривза (FR)

Направление поиска:  $d^k = -\nabla f(x^k) + w^{k-1} d^{k-1}$ , где  $w^{k-1} = \frac{\|\nabla f(x^k)\|^2}{\|\nabla f(x^{k-1})\|^2}$ .
"""

def fletcher_reeves(f, grad, x0, eps=1e-6, eps2=1e-8, max_iter=1000):
    """Реализация метода сопряженных градиентов Флетчера–Ривза."""
    x = np.array(x0, dtype=float) # Текущая точка
    history = [x.copy()] # История точек для визуализации
    g = grad(x) # Текущий градиент
    d = -g # Начальное направление антиградиента
    prev_close = False # Флаг для критерия остановки
    f_prev = f(x) # Предыдущее значение функции для критерия остановки

    for k in range(max_iter):
        if np.linalg.norm(g) < eps: # Критерий остановки по норме градиента

```

```

        break

    a = line_search(f, x, d) # Одномерный поиск оптимального шага альфа
    x_new = x + a*d # Обновление точки
    g_new = grad(x_new) # Новый градиент
    f_new = f(x_new) # Новое значение функции

    # Формула Флетчера-Ривза для обновления коэффициента бета
    beta = np.dot(g_new, g_new) / np.dot(g, g)
    d = -g_new + beta*d # Новое направление поиска

    # Критерий остановки по изменению точки и значению функции
    close = (np.linalg.norm(x_new - x) < eps and abs(f_new - f_prev) <
eps2)

    x, g, f_prev = x_new, g_new, f_new # Обновление текущих значений
    history.append(x.copy()) # Запись точки в историю
    if close and prev_close: # Если два последовательных шага были
'близкими'
        break
    prev_close = close

    return x, history

"""## Метод Полака-Рибьера (PR)

Отличается формулой  $w^{k-1}$  и обновлением (рестартом наискорейшего спуска)
каждые  $n$  шагов:

$$w^{k-1} = \frac{\langle \nabla f(x^k), \nabla f(x^k) - \nabla f(x^{k-1}) \rangle}{\|\nabla f(x^{k-1})\|^2}$$

"""

def polak_ribiere(f, grad, x0, eps=1e-6, eps2=1e-8, max_iter=1000):
    """Реализация метода сопряженных градиентов Полака-Рибьера."""
    x = np.array(x0, dtype=float)
    n = len(x) # Размерность пространства
    history = [x.copy()]
    g = grad(x)
    d = -g
    prev_close = False
    f_prev = f(x)

    for k in range(max_iter):
        if np.linalg.norm(g) < eps:
            break

        a = line_search(f, x, d)
        x_new = x + a*d
        g_new = grad(x_new)
        f_new = f(x_new)

        # Формула Полака-Рибьера для обновления бета с рестартом
        if (k+1) % n == 0: # Рестарт каждые n шагов (сброс к методу
наискорейшего спуска)
            beta = 0.0
        else:
            # beta = max(0.0, ...) для обеспечения спуска (положительная
определенность)
            beta = max(0.0, np.dot(g_new, g_new - g) / np.dot(g, g))

        d = -g_new + beta*d
        close = (np.linalg.norm(x_new - x) < eps and abs(f_new - f_prev) <
eps2)

```

```

        x, g, f_prev = x_new, g_new, f_new
        history.append(x.copy())
        if close and prev_close:
            break
        prev_close = close

    return x, history

"""## Метод DFP (Девидон-Флетчер-Пауэлл)

Хранит матрицу $G^k$ – приближение к $H^{-1}$. Обновление:

$$G^{k+1} = G^k + \frac{\Delta x^k (\Delta x^k)^T}{(\Delta x^k)^T \Delta x^k} - \frac{G^k \Delta g^k (\Delta g^k)^T G^k}{(\Delta g^k)^T G^k \Delta g^k}$$


Сброс матрицы к $E$ каждые $n$ шагов.
"""

def dfp(f, grad, x0, eps=1e-6, eps2=1e-8, max_iter=1000):
    """Реализация квазиньютоновского метода DFP."""
    x = np.array(x0, dtype=float)
    n = len(x) # Размерность пространства
    G = np.eye(n) # Начальное приближение к обратной матрице Гессе (единичная
матрица)
    history = [x.copy()]
    g = grad(x)
    prev_close = False
    f_prev = f(x)

    for k in range(max_iter):
        if np.linalg.norm(g) < eps:
            break

        d = -G @ g # Направление поиска
        a = line_search(f, x, d) # Одномерный поиск
        x_new = x + a*d
        g_new = grad(x_new)
        f_new = f(x_new)

        s = (x_new - x).reshape(-1, 1) # Вектор изменения положения (Δx)
        y = (g_new - g).reshape(-1, 1) # Вектор изменения градиента (Δg)

        # Обновление матрицы G (с рестартом каждые n шагов)
        if (k+1) % n == 0: # Рестарт: сброс G к единичной матрице
            G = np.eye(n)
        else:
            sy = (s.T @ y).item() # s^T * y
            Gy = G @ y
            yGy = (y.T @ Gy).item() # y^T * G * y
            # Проверка на деление на ноль
            if abs(sy) > 1e-14 and abs(yGy) > 1e-14:
                G = G + (s @ s.T)/sy - (Gy @ Gy.T)/yGy # Формула DFP обновления
G

        close = (np.linalg.norm(x_new - x) < eps and abs(f_new - f_prev) <
eps2)

        x, g, f_prev = x_new, g_new, f_new
        history.append(x.copy())
        if close and prev_close:
            break
        prev_close = close

    return x, history

```

```

"""## Метод BFGS (Бройден-Флетчер-Гольдфарб-Шенно)

Используем устойчивую форму обновления:

$$H^{k+1} = (I - \rho_k s_k y_k^T) H^k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$


$$\rho_k = \frac{1}{y_k^T s_k}$$

"""

def bfgs(f, grad, x0, eps=1e-6, eps2=1e-8, max_iter=1000):
    """Реализация квазиньютоновского метода BFGS."""
    x = np.array(x0, dtype=float)
    n = len(x)
    H = np.eye(n) # Начальное приближение к обратной матрице Гессе (единичная
матрица)
    history = [x.copy()]
    g = grad(x)
    prev_close = False
    f_prev = f(x)
    I = np.eye(n) # Единичная матрица для формулы обновления

    for k in range(max_iter):
        if np.linalg.norm(g) < eps:
            break

        d = -H @ g # Направление поиска
        a = line_search(f, x, d) # Одномерный поиск
        x_new = x + a*d
        g_new = grad(x_new)
        f_new = f(x_new)

        s = (x_new - x).reshape(-1, 1) # Δx
        y = (g_new - g).reshape(-1, 1) # Δg

        sy = (s.T @ y).item() # s^T * y
        # Обновление матрицы H
        if abs(sy) > 1e-14: # Проверка на деление на ноль
            rho = 1.0/sy
            # Устойчивая форма обновления BFGS
            H = (I - rho*(s @ y.T)) @ H @ (I - rho*(y @ s.T)) + rho*(s @ s.T)

        close = (np.linalg.norm(x_new - x) < eps and abs(f_new - f_prev) <
eps2)

        x, g, f_prev = x_new, g_new, f_new
        history.append(x.copy())
        if close and prev_close:
            break
        prev_close = close

    return x, history

"""## Метод L-BFGS (Limited-memory BFGS)

Хранит только m последних пар (s_i, y_i), направление вычисляется через
**двухпетлевую рекурсию**. Не требует O(n^2) памяти.
"""

def lbfgs(f, grad, x0, eps=1e-6, eps2=1e-8, max_iter=1000, m=5):
    """Реализация квазиньютоновского метода L-BFGS (Limited-memory BFGS)."""
    x = np.array(x0, dtype=float)
    history = [x.copy()]
    g = grad(x)
    s_list, y_list = [], [] # Списки для хранения последних m пар (s, y)

```



```

prev_close = False
f_prev = f(x)

for k in range(max_iter):
    if np.linalg.norm(g) < eps:
        break

    # Двухпетлевая рекурсия для вычисления направления d
    q = g.copy()
    alphas = []
    # Первая петля: проход назад по s_list и y_list
    for s_i, y_i in zip(reversed(s_list), reversed(y_list)):
        rho_i = 1.0 / np.dot(y_i, s_i)
        alpha_i = rho_i * np.dot(s_i, q)
        alphas.append(alpha_i)
        q = q - alpha_i * y_i

    # Начальное приближение H_0 (скалярное) на основе последнего s и y
    if s_list:
        gamma = np.dot(s_list[-1], y_list[-1]) / np.dot(y_list[-1],
y_list[-1])
    else:
        gamma = 1.0 # Если списки пусты, gamma = 1
    r = gamma * q

    # Вторая петля: проход вперед по s_list и y_list
    for (s_i, y_i), alpha_i in zip(zip(s_list, y_list),
reversed(alphas)):
        rho_i = 1.0 / np.dot(y_i, s_i)
        beta = rho_i * np.dot(y_i, r)
        r = r + s_i * (alpha_i - beta)

    d = -r # Направление поиска
    a = line_search(f, x, d) # Одномерный поиск
    x_new = x + a*d
    g_new = grad(x_new)
    f_new = f(x_new)

    s = x_new - x # Δx
    y = g_new - g # Δg

    # Обновление списков (s, y) с ограниченной памятью
    if np.dot(s, y) > 1e-14: # Проверка на условие кривизны (positive
definite Hessian)
        s_list.append(s)
        y_list.append(y)
        if len(s_list) > m: # Если количество пар превышает m, удаляем
старейшую
            s_list.pop(0); y_list.pop(0)

    close = (np.linalg.norm(x_new - x) < eps and abs(f_new - f_prev) <
eps2)

    x, g, f_prev = x_new, g_new, f_new
    history.append(x.copy())
    if close and prev_close:
        break
    prev_close = close

return x, history

"""## Запуск всех методов

```

```

Стартовая точка:  $x^0 = (-1.2, 1.0)$  (классическая стартовая точка для
функции Розенброка). Замеряем количество итераций и время работы.
"""

# Стартовая точка для всех методов (классическая для функции Розенброка)
x0 = np.array([-1.2, 1.0, -1.0])
# Истинный минимум функции Розенброка для N=3
x_star = np.array([1.0, 1.0, 1.0])

# Словарь с названиями методов и соответствующими функциями
methods = {
    'Флетчер-Ривз': fletcher_reeves,
    'Полак-Рибьер': polak_ribiere,
    'DFP': dfp,
    'BFGS': bfgs,
    'L-BFGS': lbfgs,
}

results = {} # Словарь для хранения результатов каждого метода
# Запуск каждого метода, измерение времени и сохранение результатов
for name, method in methods.items():
    t0 = time.perf_counter() # Начало отсчета времени
    x_found, history = method(rosen, rosen_grad, x0, eps=1e-6, max_iter=2000)
    elapsed = time.perf_counter() - t0 # Затраченное время
    results[name] = {
        'x': x_found, # Найденная точка минимума
        'f': rosen(x_found), # Значение функции в найденной точке
        'iters': len(history) - 1, # Количество итераций
        'time': elapsed, # Время выполнения
        'err': np.linalg.norm(x_found - x_star), # Ошибка (расстояние до
истинного минимума)
        'history': np.array(history), # Полная траектория поиска
    }

# Создание сводной таблицы результатов с помощью pandas DataFrame
summary = pd.DataFrame([
    {
        'Метод': name,
        'Итераций': r['iters'],
        'x1*': r['x'][0],
        'x2*': r['x'][1],
        'f(x*)': r['f'],
        '||x*-x||': r['err'],
        'Время, мс': r['time'] * 1000, # Перевод времени в миллисекунды
    } for name, r in results.items()
])

# Вывод таблицы с форматированием чисел для лучшей читаемости
print(summary.to_string(float_format="{:.6f}".format))

"""## Траектории поиска на линиях уровня

Видно, как каждый метод пробирается по «банановому оврагу» функции Розенброка.
"""

# Создание подграфиков для каждого метода (2 ряда, 3 колонки)
fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.flatten() # Преобразование 2D массива осей в 1D для удобства
итерации

# Итерация по результатам каждого метода для построения траекторий
for ax, (name, r) in zip(axes, results.items()):
    # Построение линий уровня функции Розенброка на каждом подграфике

```

```

ax.contour(X1, X2, Z, levels=levels, cmap='viridis', alpha=0.6)
h = r['history'] # Траектория поиска для текущего метода
# Построение траектории: точки и линии, соединяющие их
ax.plot(h[:, 0], h[:, 1], '-o', color='red', markersize=3, linewidth=1,
label='траектория')
# Отметка стартовой точки
ax.plot(*x0[:2], 'bs', markersize=10, label='старт') # x0[:2] потому что
N=3, а график 2D
# Отметка точки минимума
ax.plot(1, 1, 'g*', markersize=15, label='минимум')
# Заголовок подграфика с названием метода и количеством итераций
ax.set_title(f"{name} (итераций: {r['iters']})")
ax.set_xlabel('x1'); ax.set_ylabel('x2') # Подписи осей
ax.legend(loc='upper left', fontsize=8) # Легенда

# Отключение последнего (пустого) подграфика, если методов меньше, чем
подграфиков
axes[-1].axis('off')
plt.tight_layout(); plt.show() # Оптимизация расположения и отображение

"""## Скорость сходимости

```

Строим график $\|\nabla f(x^k)\|$ от номера итерации в логарифмическом масштабе. Чем круче падение – тем быстрее сходимость.

```

fig, ax = plt.subplots(figsize=(9, 5))
# Построение графиков сходимости по норме градиента для каждого метода
for name, r in results.items():
    # Вычисление нормы градиента для каждой точки в истории поиска
    grads = [np.linalg.norm(rosen_grad(p)) for p in r['history']]
    # Построение графика в логарифмическом масштабе по оси Y
    ax.semilogy(grads, label=f"{name} ({r['iters']} ит.)")
# Добавление горизонтальной линии, показывающей уровень точности (epsilon)
ax.axhline(1e-6, color='gray', linestyle='--', linewidth=0.7, label='ε = 1e-
6')

ax.set_xlabel('Номер итерации') # Подпись оси X
ax.set_ylabel('||∇f(x_k)||') # Подпись оси Y
ax.set_title('Сходимость по норме градиента') # Заголовок графика
ax.legend(); ax.grid(True, alpha=0.3) # Легенда и сетка
plt.tight_layout(); plt.show() # Оптимизация расположения и отображение

# График сходимости по значению функции (f - f*)
fig, ax = plt.subplots(figsize=(9, 5))
# Построение графиков сходимости по значению функции для каждого метода
for name, r in results.items():
    # Вычисление разницы между значением функции в текущей точке и истинным
минимумом
    fvals = [rosen(p) - F0 for p in r['history']]
    # Построение графика в логарифмическом масштабе по оси Y
    ax.semilogy(fvals, label=f"{name}")
ax.set_xlabel('Номер итерации') # Подпись оси X
ax.set_ylabel('f(x_k) - f*') # Подпись оси Y
ax.set_title('Сходимость по значению функции') # Заголовок графика
ax.legend(); ax.grid(True, alpha=0.3) # Легенда и сетка
plt.tight_layout(); plt.show() # Оптимизация расположения и отображение

"""## Выводы

```

1. **Стационарная точка единственна:** $x^* = (1, 1, 1)$, $f(x^*) = 30$. Гессиан в ней положительно определён → это глобальный минимум.

2. ****Все 5 методов сошлись**** к одной и той же точке с заданной точностью $\epsilon = 10^{-6}$.

3. ****Скорость сходимости**** (по числу итераций, как правило):

- Квазиньютоновские методы (BFGS, DFP, L-BFGS) сходятся быстрее – суперлинейная скорость.
- Методы сопряжённых градиентов (FR, PR) требуют больше итераций – линейная/суперлинейная скорость.
- Полак–Рибьер обычно стабильнее Флетчера–Ривза для неквадратичных функций.

4. ****Особенность функции Розенброка**** при $a = 70$ – очень узкий и крутой овраг, поэтому простые методы могут «зигзагить»; квазиньютоновские методы выпрямляют траекторию за счёт аппроксимации обратного гессиана.

"""